Treći popravni test iz
Algoritama i struktura podataka

Prvi praktični test održan 06.03.2026.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test1_GrupaA_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Quick Sort*

Napisati ***quick*** metodu ***Quick Sort*** algoritma za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomNumberGen*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***QuickSort*** napisati implementaciju za metodu ***private static void quickSort(int[] arr, int lower, int upper)***
- Obezbediti kod u **Main** metodi za testiranje algoritma.

Zadatak 2 – *Izbacivanje elemenata van parnih granica*

Implementirati metodu koja u jednom prolazu kroz povezanu listu pronalazi prvi i poslednji parni broj. Metoda treba da **obriše** sve elemente koji se nalaze **pre** prvog parnog broja i sve elemente koji se nalaze **posle** poslednjeg parnog broja. Ako u listi nema parnih brojeva, lista treba da postane prazna.

Dodatni zahtevi za rešavanje zadatka:

- Ako minimalna vrednost ne postoji (prazna lista), vratiti 0.
- Algoritam pisati u okviru klase ***LinkedList.***
- Algoritam mora imati vremensku kompleksnost **O(n)**.
- Zahtevan potpis metode: ***public void trimToEvenBounds ()***

Primer:

- Input: 1 → 3 → 4 → 5 → 6 → 7 ;
- Output: 4 → 5 → 6

Zadatak 3 – *Monitoring sistema za keširanje podataka (Cache Loss Tracker)*

Razvijate sistem za keširanje koji skladišti podatke različitih veličina (u megabajtima). Keš memorija ima strogo ograničen kapacitet. Podaci se upisuju redom, a sistem funkcioniše po principu "najstariji prvi napolje" (**FIFO**).

Opis zadatka: Napisati metodu koja prima niz celih brojeva dataSizes (veličine podataka koji pristižu) i cacheCapacity (maksimalni kapacitet keša).

1. Podaci se dodaju u keš jedan po jedan.
2. Pre nego što se podatak doda, mora se osloboditi dovoljno prostora brisanjem najstarijih podataka.
3. Ako je sam podatak koji pristiže veći od ukupnog kapaciteta keša, on se odbacuje i smatra se "izgubljenim".
4. Metoda treba da izračuna i vrati **ukupnu količinu podataka (u MB) koji su trajno obrisani ili odbačeni** tokom procesa.

Dodatni zahtevi za rešavanje zadatka:

1. Povratna vrednost: int
2. Koristiti pomoćnu strukturu reda (***Queue***) koja je data u šablonu.
3. Algoritam pisati u okviru metode: ***public int calculateTotalDataLoss(int[] dataSizes, int cacheCapacity)***

Primer:

Input 1: dataSizes = [10, 20, 30, 5, 45], cacheCapacity = 50

Simulacija korak po korak:

1. Stiže **10**: Keš (10/50). Loss = 0.
2. Stiže **20**: Keš (10, 20 / Sum: 30/50). Loss = 0.
3. Stiže **30**: Premašuje 50 (30+30=60). Izbacuje se najstariji (**10**). Keš (20, 30 / Sum: 50/50). **Loss = 10**.
4. Stiže **5**: Premašuje 50 (50+5=55). Izbacuje se najstariji (**20**). Keš (30, 5 / Sum: 35/50). **Loss = 10 + 20 = 30**.
5. Stiže **45**: Premašuje 50 (35+45=80). Izbacuje se najstariji (**30**), pa sledeći (**5**). Keš (45 / Sum: 45/50). **Loss = 30 + 30 + 5 = 65**.

Output 1: 65